
Data Structures

BS (CS) _Fall_2025

Lab_06 Task



Learning Objectives:

1. Linked Lists (Doubly, Circular)

Lab Task 1: Music Playlist Manager

Scenario

You are developing a music player application that needs to manage playlists efficiently. Users should be able to navigate through songs in both directions and add songs at any position.

Task Requirements

Create a doubly linked list where each node contains:

- Song Title (string)
- Artist Name (string)
- Duration (in seconds, integer)
- Pointer to next song
- Pointer to previous song

Core Functionalities to Implement

1. Playlist Management:

- `addSongAtBeginning(title, artist, duration)` - Add a new song at the start
- `addSongAtEnd(title, artist, duration)` - Add a new song at the end
- `addSongAtPosition(position, title, artist, duration)` - Insert song at specific position
- `removeSong(title)` - Remove a song by title
- `removeSongAtPosition(position)` - Remove song at specific position

2. Navigation System:

- `std::string displayCurrentSong():` returns "Title by Artist", returns "No song" if playlist is empty.
- `moveToNext()` - Move to next song in playlist
- `moveToPrevious()` - Move to previous song in playlist
- `displayPlaylist()` - Show all songs with position numbers

3. Advanced Features:

- `getTotalDuration()` - Calculate total playlist duration
- `findSong(title)` - Search for a song and return its position

4. Implementation Guidelines

- Implement a Playlist class with appropriate member functions
- Handle edge cases (empty playlist, single song, invalid positions)

- Use proper memory management (constructor, destructor, copy constructor)

Test cases:

```
// 1. Add at beginning
TEST(PlaylistTest, AddAtBeginning) {
    Playlist p;
    p.addSongAtBeginning("Song A", "Artist 1", 200);
    EXPECT_EQ(p.findSong("Song A"), 1);
}

// 2. Add at end
TEST(PlaylistTest, AddAtEnd) {
    Playlist p;
    p.addSongAtEnd("Song B", "Artist 2", 150);
    EXPECT_EQ(p.findSong("Song B"), 1);
}

// 3. Add at position (check return of findSong)
TEST(PlaylistTest, AddAtPosition) {
    Playlist p;
    p.addSongAtEnd("A", "X", 100);
    p.addSongAtEnd("B", "Y", 200);
    p.addSongAtEnd("C", "Z", 300);
    p.addSongAtPosition(2, "D", "W", 250);
    EXPECT_EQ(p.findSong("D"), 2);
}

// 4. Remove by title
TEST(PlaylistTest, RemoveSong) {
    Playlist p;
    p.addSongAtEnd("Song A", "Artist 1", 200);
    p.addSongAtEnd("Song B", "Artist 2", 150);
    p.removeSong("Song A");
    EXPECT_EQ(p.findSong("Song A"), -1);
    EXPECT_EQ(p.findSong("Song B"), 1);
}

// 5. Remove by position
TEST(PlaylistTest, RemoveAtPosition) {
    Playlist p;
    p.addSongAtEnd("X", "Y", 100);
    p.addSongAtEnd("Y", "Z", 200);
    p.removeSongAtPosition(1);
    EXPECT_EQ(p.findSong("X"), -1);
}
```

```

    EXPECT_EQ(p.findSong("Y"), 1);
}

// 6. Move to next (just check that current pointer moves logically)
TEST(PlaylistTest, MoveNext) {
    Playlist p;
    p.addSongAtEnd("A", "X", 100);
    p.addSongAtEnd("B", "Y", 200);
    p.moveToNext();
    EXPECT_EQ(p.findSong("B"), 2); // still in playlist
}

// 7. Move to previous
TEST(PlaylistTest, MovePrevious) {
    Playlist p;
    p.addSongAtEnd("A", "X", 100);
    p.addSongAtEnd("B", "Y", 200);
    p.moveToNext();
    p.moveToPrevious();
    EXPECT_EQ(p.findSong("A"), 1);
}

// 8. FindSong
TEST(PlaylistTest, FindSong) {
    Playlist p;
    p.addSongAtEnd("Alpha", "X", 100);
    EXPECT_EQ(p.findSong("Alpha"), 1);
    EXPECT_EQ(p.findSong("Beta"), -1);
}

// 9. Total duration
TEST(PlaylistTest, TotalDuration) {
    Playlist p;
    p.addSongAtEnd("A", "X", 100);
    p.addSongAtEnd("B", "Y", 200);
    EXPECT_EQ(p.getTotalDuration(), 300);
}

// 10. Remove non-existent
TEST(PlaylistTest, RemoveNonExistent) {
    Playlist p;
    p.addSongAtEnd("A", "X", 100);
    p.removeSong("B");
    EXPECT_EQ(p.findSong("A"), 1);
}

```

```
// 11. Remove invalid position
TEST(PlaylistTest, RemoveInvalidPosition) {
    Playlist p;
    p.addSongAtEnd("A", "X", 100);
    p.removeSongAtPosition(10);
    EXPECT_EQ(p.findSong("A"), 1);
}

// 12. Multiple operations
TEST(PlaylistTest, MultipleOperations) {
    Playlist p;
    p.addSongAtBeginning("A", "X", 100);
    p.addSongAtEnd("B", "Y", 200);
    p.addSongAtEnd("C", "Z", 300);
    p.removeSong("B");
    EXPECT_EQ(p.findSong("B"), -1);
    EXPECT_EQ(p.findSong("A"), 1);
    EXPECT_EQ(p.findSong("C"), 2);
}
```

Lab Task 2: Circular Linked List Implementation

Scenario

You are tasked with implementing a fundamental circular linked list data structure from scratch. This implementation will serve as the foundation for more complex circular data structures used in applications like round-robin scheduling, circular buffers, and continuous media playback systems.

Task Requirements

Implement a Node and CircularLinkedList Class

Core Functionalities to Implement

- CircularLinkedList() - Default constructor to initialize empty list
- insert(data) - Insert data item at the end of circular linked list
 - If list is empty, new node becomes head and points to itself
 - If list has elements, insert after the last node and maintain circularity
- isEmpty() - Check if the circular linked list is empty
- print() - Display all elements starting from head until reaching head again

Search and Update Operations:

- search(data) - Search for specific element, return true if found, false otherwise
- update(oldData, newData) - Replace/update one data element with another
- insertAtIndex(index, data) - Insert new element at specified position (0-indexed)
 - Count elements in the list to determine valid positions
 - Handle edge cases for index 0 and end positions

Deletion Operations:

- deleteData(data) - Delete the first occurrence of specified data element
- Handle special cases: single node, head node deletion, last node deletion

Test Cases:

```
// 1. Empty list
TEST(CircularListTest, IsEmptyInitially) {
    CircularLinkedList cl;
    EXPECT_TRUE(cl.isEmpty());
    EXPECT_EQ(cl.size(), 0);
}

// 2. Insert first element
TEST(CircularListTest, InsertFirstElement) {
    CircularLinkedList cl;
    cl.insert(10);
    EXPECT_FALSE(cl.isEmpty());
    EXPECT_EQ(cl.size(), 1);
    EXPECT_EQ(cl.getAtIndex(0), 10);
}
```

```

// 3. Insert multiple elements
TEST(CircularListTest, InsertMultipleElements) {
    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);
    EXPECT_EQ(cl.size(), 3);
    EXPECT_EQ(cl.getAtIndex(2), 3);
}

// 4. Search existing element
TEST(CircularListTest, SearchElement) {
    CircularLinkedList cl;
    cl.insert(5);
    cl.insert(15);
    cl.insert(25);
    EXPECT_TRUE(cl.search(15));
    EXPECT_FALSE(cl.search(99));
}

// 5. Update element
TEST(CircularListTest, UpdateElement) {
    CircularLinkedList cl;
    cl.insert(10);
    cl.insert(20);
    EXPECT_TRUE(cl.update(20, 25));
    EXPECT_TRUE(cl.search(25));
    EXPECT_FALSE(cl.update(99, 100));
}

// 6. Insert at index 0 (head)
TEST(CircularListTest, InsertAtHead) {
    CircularLinkedList cl;
    cl.insert(5);
    cl.insert(10);
    cl.insertAtIndex(0, 99);
    EXPECT_EQ(cl.getAtIndex(0), 99);
}

// 7. Insert at middle
TEST(CircularListTest, InsertAtMiddle) {
    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);

```

```

        cl.insertAtIndex(1, 50);
        EXPECT_EQ(cl.getAtIndex(1), 50);
        EXPECT_EQ(cl.size(), 4);
    }

    // 8. Insert at end
    TEST(CircularListTest, InsertAtEnd) {
        CircularLinkedList cl;
        cl.insert(10);
        cl.insert(20);
        cl.insertAtIndex(2, 30);
        EXPECT_EQ(cl.getAtIndex(2), 30);
    }

    // 9. Insert invalid index
    TEST(CircularListTest, InsertInvalidIndex) {
        CircularLinkedList cl;
        cl.insert(1);
        EXPECT_FALSE(cl.insertAtIndex(5, 99));
    }

    // 10. Delete head element
    TEST(CircularListTest, DeleteHead) {
        CircularLinkedList cl;
        cl.insert(10);
        cl.insert(20);
        cl.insert(30);
        EXPECT_TRUE(cl.deleteData(10));
        EXPECT_FALSE(cl.search(10));
    }

    // 11. Delete last element
    TEST(CircularListTest, DeleteLast) {
        CircularLinkedList cl;
        cl.insert(10);
        cl.insert(20);
        cl.insert(30);
        EXPECT_TRUE(cl.deleteData(30));
        EXPECT_EQ(cl.size(), 2);
    }

    // 12. Delete middle element
    TEST(CircularListTest, DeleteMiddle) {
        CircularLinkedList cl;
        cl.insert(1);

```



```
        cl.insert(2);
        cl.insert(3);
        EXPECT_TRUE(cl.deleteData(2));
        EXPECT_FALSE(cl.search(2));
    }

    // 13. Delete only element
    TEST(CircularListTest, DeleteSingleNode) {
        CircularLinkedList cl;
        cl.insert(100);
        EXPECT_TRUE(cl.deleteData(100));
        EXPECT_TRUE(cl.isEmpty());
    }

    // 14. Delete non-existing element
    TEST(CircularListTest, DeleteNonExisting) {
        CircularLinkedList cl;
        cl.insert(10);
        EXPECT_FALSE(cl.deleteData(999));
    }

    // 15. Print list (manual visual check)
    TEST(CircularListTest, PrintList) {
        CircularLinkedList cl;
        cl.insert(1);
        cl.insert(2);
        cl.insert(3);
        cl.print(); // Just call (visual verification)
    }
}
```